Workshop No. 2

System Design Document

Academic Collaboration Network

Skill Matching & Resource Sharing Platform

-

Team 8

Silva González Kevin Santiago · Casas Zapata Anyelo Esteban
Beltrán Varela Gabriel Andrés · Tarazona Correa Miguel David

Systems Analysis & Design — Semester 2026-I

Eng. Carlos Andrés Sierra, M.Sc.

Universidad Distrital Francisco José de Caldas

March 2026

Abstract

This document presents a comprehensive system design for the Academic Collaboration Network, addressing critical collaboration barriers identified in Workshop #1's primary data analysis ($n = 25$, March 2026). The design employs a **microservices architecture** with seven core services, achieving scalability to 500+ concurrent users, <2 s response times, and 99.5% availability. Key innovations include an **intelligent skill matching engine** addressing the 36% communication barrier, seamless **LMS/library integration** resolving the 88% WhatsApp dependency, and robust security protocols for high-sensitivity components. The implementation roadmap spans 7 months across three phases: MVP foundation, integration & enhancement, and optimization & scale.

Contents

1 Executive Summary **4**
- 1.1 Problem Statement and Context . . . 4
- 1.2 Design Approach . . . 4
- 1.3 Key Design Decisions . . . 4
- 1.4 Expected Outcomes . . . 4

2 Requirements Analysis **4**
- 2.1 Functional Requirements . . . 5
- 2.2 Non-Functional Requirements . . . 8
- 2.3 Traceability Matrix . . . 8

3 Requirements Traceability **9**

4 System Architecture Data Design **9**
- 4.1 Microservices Summary . . . 9

5 Strategy Risk Management **11**
- 5.1 Implementation Roadmap (7 Months) . . . 11
- 5.2 Critical Risk Matrix . . . 11

6 Performance and Optimization **11**
- 6.1 Performance Requirements . . . 11
- 6.2 Optimization Strategies . . . 11
- 6.3 Monitoring and Observability . . . 12
- 6.4 Scalability Planning . . . 12

7 Quality Assurance and Testing **14**
- 7.1 Testing Strategy . . . 14
- 7.2 Security Testing . . . 14
- 7.3 User Acceptance Testing . . . 14
- 7.4 CI/CD Pipeline . . . 14

8 Conclusion and Next Steps **15**
- 8.1 Design Summary . . . 15
- 8.2 Expected Impact . . . 15
- 8.3 Immediate Next Steps . . . 16
- 8.4 Future Workshops . . . 16
- 8.5 Continuous Improvement . . . 16

8.6 Acknowledgments 16
References **17**

1 Executive Summary

1.1 Problem Statement and Context

Workshop #1's systems analysis revealed critical collaboration barriers: **88% of students rely exclusively on WhatsApp** with zero institutional integration, **36% identify lack of communication** as the primary obstacle to group formation, and no systematic mechanism exists for skill-based matching. Despite these challenges, **96% expressed interest** in a dedicated platform (average likelihood 3.9/5), validating strong market demand.

1.2 Design Approach

The Academic Collaboration Network employs a **microservices architecture** organized into seven services: (1) User Profile Service, (2) Skill Matching Engine **[HIGH sensitivity]**, (3) Group Workspace Service, (4) Notification Engine, (5) Authentication & Authorization **[HIGH sensitivity]**, (6) Analytics Dashboard, and (7) Integration Gateway. This architecture enables independent scaling, fault isolation, and technology flexibility while addressing all functional and non-functional requirements identified in Workshop #1.

1.3 Key Design Decisions

Table 1: *Critical Architectural Decisions*

Decision	Justification	Trade-off
Microservices	Independent scaling; fault isolation	Deployment complexity
PostgreSQL + Redis	ACID compliance + caching; proven reliability	Dual-database operations
React + React Native	70% code reusability; strong ecosystem	Framework lock-in
OAuth 2.0 + JWT	Industry-standard; SSO compatibility	Token management overhead
Phased Deployment	Early feedback; risk mitigation	Extended timeline

1.4 Expected Outcomes

- **Communication Enhancement:** Reduce communication barriers from 36% to <10% through integrated notifications and workspaces.
- **Platform Consolidation:** Transition from 88% WhatsApp-only to 80%+ unified platform adoption.
- **Matching Quality:** >80% participant satisfaction with algorithmic group recommendations.
- **Performance:** 500+ concurrent users with <2 s response times (p95).
- **Reliability:** 99.5% uptime with automated failover and monitoring.

2 Requirements Analysis

This section translates Workshop #1 findings into measurable design requirements with full traceability.

2.1 Functional Requirements

FR1 — User Profile Management

Complete profile creation (<5 min) with skills, availability, preferences, and privacy controls. Profile completeness scoring to encourage comprehensive data entry.

FR2 — Intelligent Skill Matching

Multi-dimensional algorithm analyzing skill complementarity, academic alignment, schedule compatibility, and learning styles. Match results in <3 s with explainability (e.g., "80% compatible: shared Calculus II interest, complementary problem-solving skills"). Target: >80% user satisfaction.

FR3 — Group Workspace

Real-time messaging, file sharing (50 MB limit), task boards, meeting scheduler, and resource library linkage. Directly addresses the 88% WhatsApp dependency.

FR4 — Multi-Channel Notifications

Email, push, and in-app alerts for matches, invitations, messages, and deadlines. User-configurable preferences with do-not-disturb support. Delivery rate >95%.

FR5 — LMS & Library Integration

OAuth SSO, automated course sync (daily), grade import with consent, library resource search, and real-time availability checking.

FR6 — Analytics Dashboard

Real-time metrics (DAU, match conversion, group formation), algorithm performance tracking, and administrative reporting.

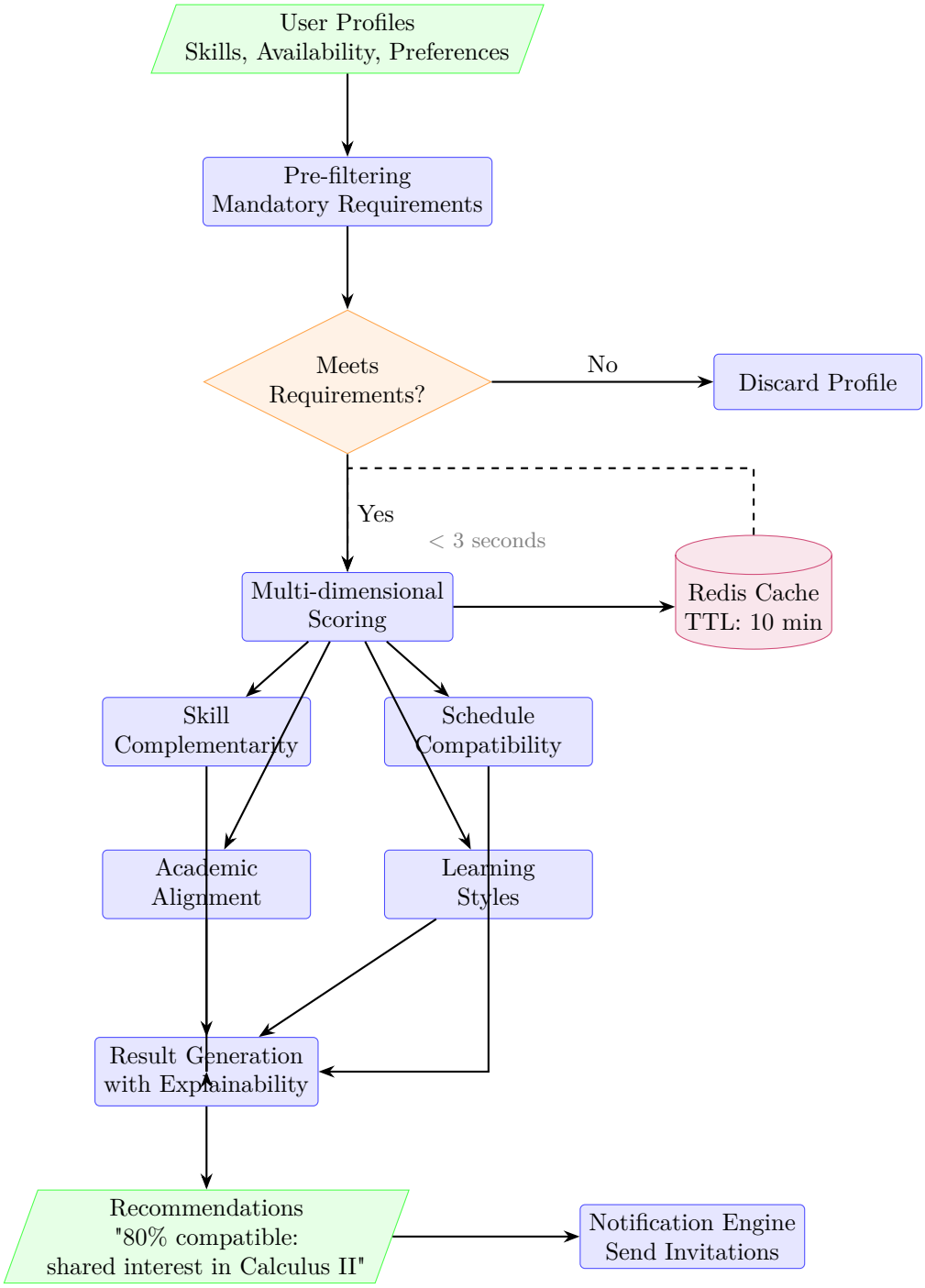

Figure 1: *Skill Matching Engine Processing Flow*

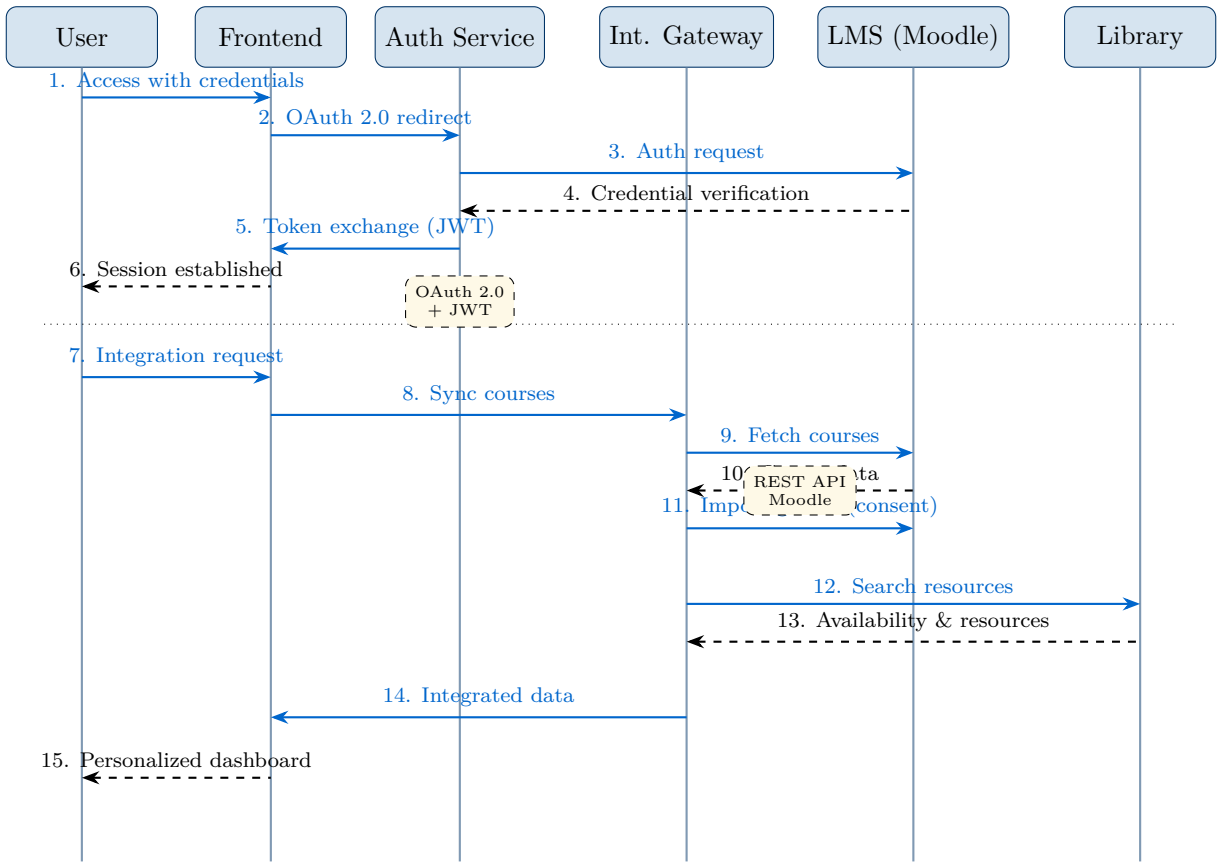

Figure 2: *Sequence Diagram: SSO Authentication and LMS/Library Integration*

2.2 Non-Functional Requirements

Table 2: *Non-Functional Requirements Specification*

Category	Requirement	Target
Performance	Page load, API response, matching latency	<2 s (p95) / <500 ms / <3 s
Reliability	System uptime, MTTR, data durability	99.5% / <15 min / zero loss
Security	Auth, encryption, privacy compliance	OAuth+MFA, TLS 1.3, GDPR
Scalability	Concurrent users, horizontal scaling	500+ / linear to 5,000+
Usability	Onboarding time, accessibility, i18n	<5 min / WCAG 2.1 AA / ES+EN
Maintainability	Code coverage, documentation, CI/CD	80% / OpenAPI / automated

2.3 Traceability Matrix

Table 3: *Workshop #1 Findings to Design Requirements Traceability*

Workshop #1 Finding	Requirement	Priority
88% WhatsApp dependency (Survey Q5)	FR3, FR5	**Critical**
36% communication barrier (Survey Q4)	FR4	**Critical**
Skill Matcher HIGH sensitivity (Diagram 4)	FR2, NFR-Sec	**Critical**
Auth HIGH sensitivity (Diagram 4)	NFR-Security	**Critical**
96% platform interest (Survey Q6)	NFR-Usability	**High**
No LMS/Library integration (Diagram 1)	FR5	**High**

3 Requirements Traceability

Table 4: *Unified Functional Non-Functional Requirements*

ID	Requirement Description	Target / Priority
FR1	Profile: Skills, availability	High
FR2	ML Matching: Complementarity score	Critical (<3s)
FR3	Workspace: Real-time chat & S3 Files	Critical (50MB)
FR5	Integrations: Moodle SSO & Library	High
NFR1	Performance: Load time / API response	<2s / <500ms
NFR2	Availability: Multi-AZ uptime	99.5%

4 System Architecture Data Design

Service	Core Responsibility	Tech Stack
Profile & Auth	User identity & RBAC (SSO)	Node.js / Postgre
Matching Engine	Compatibility scoring (Python)	FastAPI / Redis
Workspace	WebSockets & Messaging	Socket.io / S3
Notification	Event-driven alerts	RabbitMQ / Firebase

4.1 Microservices Summary

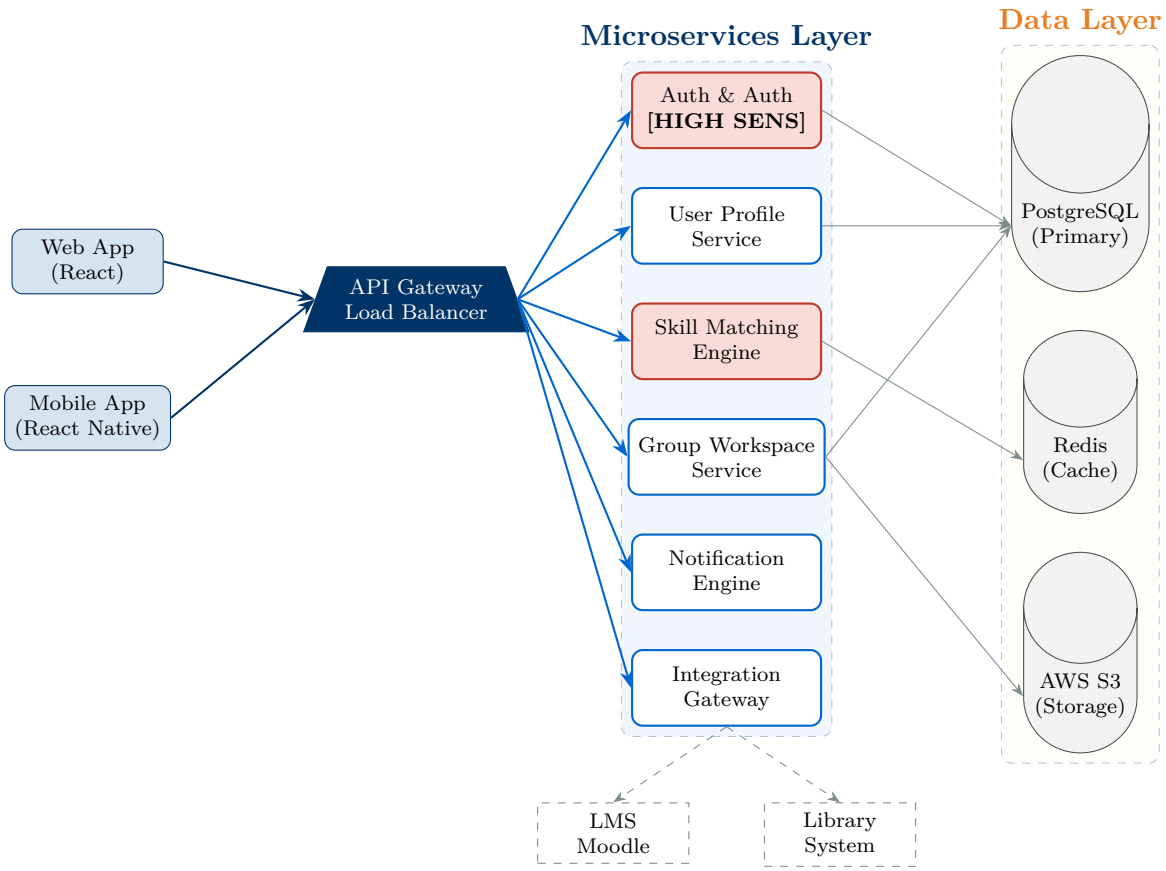

Figure 3: *Refined Microservices Architecture Overview for Academic Collaboration Network*

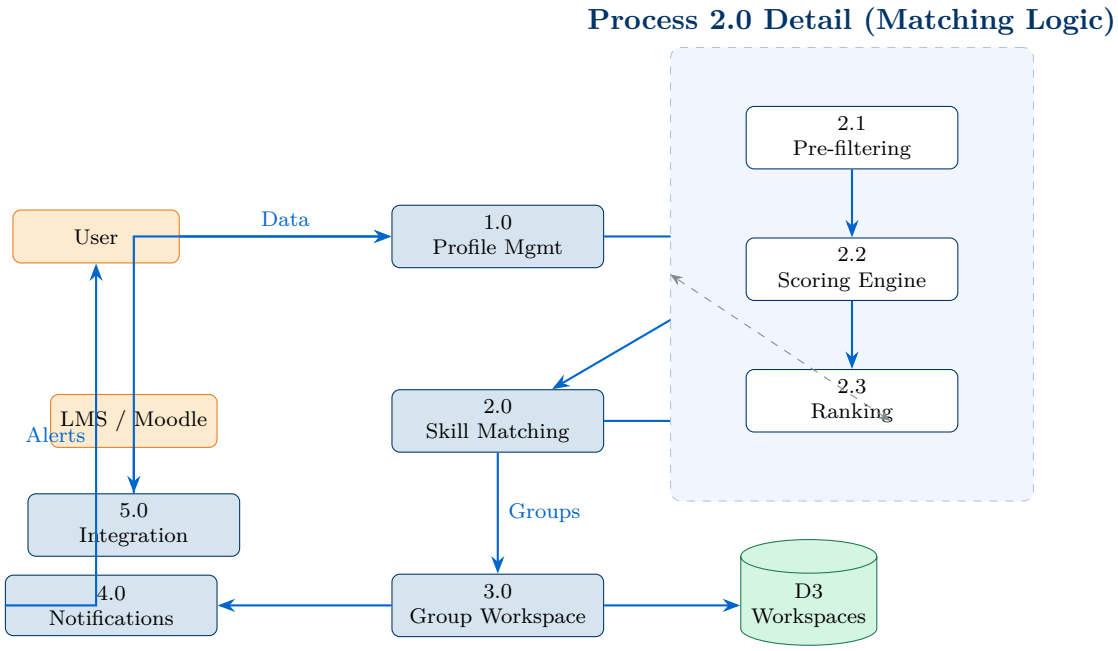

Figure 4: *Data Flow Diagram (DFD): Level 0 and Level 1 (Matching Process) zoomed detail.*

5 Strategy Risk Management

5.1 Implementation Roadmap (7 Months)

- **Phase 1 (MVP):** Auth, Profile, and Basic Matcher. (Month 1-3)
- **Phase 2 (Scale):** LMS Integration, Mobile Apps, ML v2. (Month 4-5)
- **Phase 3 (Opt):** Analytics, UX Audit, Performance tuning. (Month 6-7)

5.2 Critical Risk Matrix

Risk	Impact	Mitigation
Low Adoption	High	Gamification & Faculty Ambassadors
Algorithm Bias	Critical	Continuous A/B Testing & Manual Fallback
Security Breach	Critical	OAuth+MFA & Quarterly Pentesting

6 Performance and Optimization

6.1 Performance Requirements

Table 5: *Performance Targets and Measurement*

Metric	Target	Measurement Tool
Page Load Time	<2 s (p95)	Real User Monitoring (RUM)
API Response Time	<500 ms (p95)	APM — New Relic / Datadog
Matching Latency	<3 s	Application-level timer
Database Query	<200 ms (p95)	PostgreSQL slow query log
Concurrent Users	500+	Load testing with JMeter
Cache Hit Rate	≥90%	Redis INFO stats

6.2 Optimization Strategies

Frontend:

- Code splitting (route-based lazy loading).
- WebP image format with responsive sizes.
- Service worker caching for static assets.
- `React.memo()` for expensive components; virtual scrolling for long lists.
- Input debouncing (300 ms) for search fields.

Backend:

- Database indexing on high-frequency query columns (see Section 3.5).
- Redis caching: profiles 15 min TTL, matches 10 min, messages 5 min.
- Asynchronous processing via RabbitMQ for notifications and emails.
- API rate limiting: 100 req/min per authenticated user.

Matching Algorithm:

- Pre-filter by mandatory constraints before scoring.
- Elasticsearch for full-text skill search.
- Locality-Sensitive Hashing (LSH) for user base >5,000.
- Parallel workers for concurrent match computations.

6.3 Monitoring and Observability

Tool	Metrics Tracked
New Relic / Datadog (APM)	Request throughput, response time (p50/p95/p99), error rate, DB performance, external API latency, distributed tracing
Prometheus + Grafana (Infrastructure)	CPU / memory / disk / network, container health, DB connections, replication lag, cache hit rate
ELK Stack (Logging)	Structured JSON logs; hot storage 7 days, warm 8–30 days, archive 31–90 days

Alert Thresholds:

- **Critical:** 5xx error rate >5% · DB connections exhausted · Service down 3× health check.
- **High:** p95 latency >4 s · Cache hit rate <70% · Disk usage >85%.

6.4 Scalability Planning

Horizontal Scaling: All services are stateless; Kubernetes HPA with CPU target 70%; minimum 2 replicas per service for availability.

Database Scaling: Read replicas for analytics workloads; PgBouncer connection pooling (1,000 app → 100 DB connections); sharding roadmap by institution for future multi-tenant support.

Load Testing Scenarios:

Scenario	Load	Goal
Baseline	50 users	Establish performance baseline
Peak	500 users	Validate target capacity
Stress	Find breaking point	Identify system limits
Spike	10× surge	Validate auto-scaling response
Endurance	500 users / 24 h	Detect memory leaks and drift

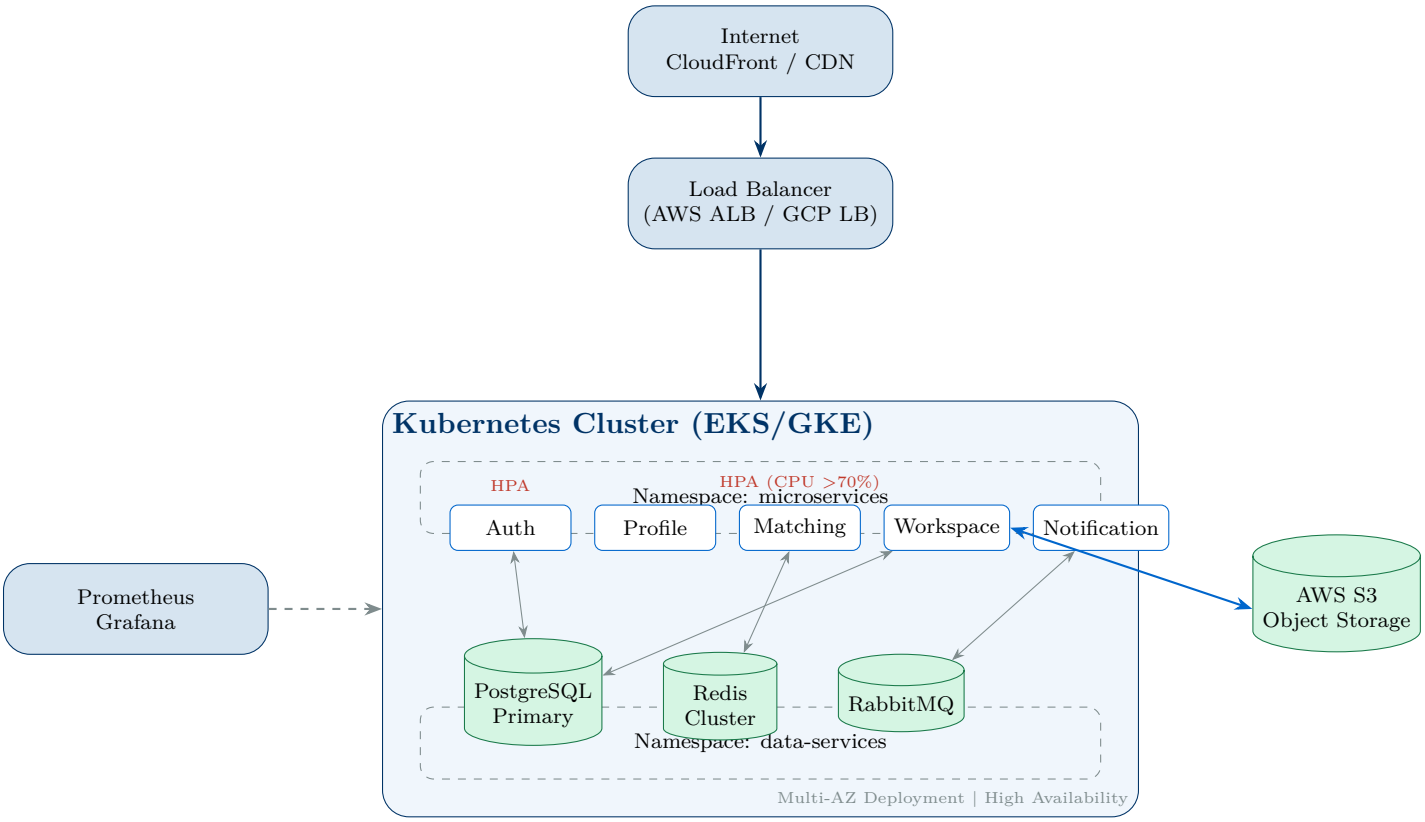

Figure 5: *Deployment Diagram: Kubernetes Infrastructure with Auto-scaling and High Availability.*

7 Quality Assurance and Testing

7.1 Testing Strategy

Table 6: *Comprehensive Testing Approach*

Test Type	Tools	Coverage / Target
Unit Testing	Jest (JS) · Pytest (Python)	80% code coverage
Integration Testing	Supertest · Postman	All API endpoints
End-to-End Testing	Cypress · Selenium	Critical user paths
Performance Testing	JMeter · k6	Key load scenarios
Security Testing	OWASP ZAP · Snyk · Burp Suite	OWASP Top 10 — quarterly
Accessibility	axe DevTools · WAVE · screen readers	WCAG 2.1 AA
User Acceptance	Manual + NPS/SUS surveys	30–50 beta users

7.2 Security Testing

Automated (continuous):

- OWASP ZAP scans — nightly against staging environment.
- Snyk vulnerability scan — triggered on every pull request.
- npm audit / pip-audit — dependency vulnerability checks.

OWASP Top 10 Validation:

- **A01 — Broken Access Control:** RBAC testing, privilege escalation attempts.
- **A02 — Cryptographic Failures:** SSL Labs scan, cipher suite audit.
- **A03 — Injection:** SQLMap automated testing.
- **A07 — Auth Failures:** Brute force simulation, account lockout verification.
- **A10 — SSRF:** Internal service access attempt testing.

Quarterly: External penetration testing by third-party security firm.

7.3 User Acceptance Testing

Process: Recruit 30–50 beta users → 30-minute guided onboarding → 2-week free exploration → post-test SUS survey + interviews.

7.4 CI/CD Pipeline

GitHub Actions Workflow:

1. Run unit tests (Jest / Pytest).
2. Upload code coverage to Codecov.
3. Snyk vulnerability scan.

Table 7: *UAT Success Criteria*

Metric	Target	Tool
SUS (System Usability Scale)	≥70 (above average)	Online survey
Task completion rate	≥80%	Moderated testing
Net Promoter Score (NPS)	≥40	Post-test survey
Critical bugs found	<10	Bug tracking system

4. Build Docker images.
5. Run integration tests (Supertest / Postman).
6. Deploy to staging environment.
7. Run E2E tests (Cypress).
8. OWASP ZAP security scan.
9. *If all pass:* deploy to production via blue-green switch.

Quality Gates (all must pass before merge): All tests pass · 80% code coverage · Zero high-severity vulnerabilities · SonarQube quality gate passed.

8 Conclusion and Next Steps

8.1 Design Summary

This comprehensive system design addresses all collaboration barriers identified in Workshop #1 through a scalable microservices architecture. The **Skill Matching Engine** directly tackles the 36% communication gap with intelligent algorithmic recommendations, while seamless **LMS/library integration** resolves the 88% WhatsApp dependency. High-sensitivity components (Matching, Auth) employ enhanced security protocols, and the phased 7-month implementation enables continuous iterative feedback from real users.

8.2 Expected Impact

Table 8: *Projected Metrics Improvement*

Metric	Baseline	Target
Platform Adoption	0%	**80%**
Communication Satisfaction	64% (36% issues)	**90%**
Match Quality Satisfaction	N/A	**≥80%**
LMS Integration Usage	0%	**70%**
Average Group Longevity	Unknown	**≥30 days**

8.3 Immediate Next Steps

1. **Stakeholder Approval:** Present design to university IT department, academic advisors, and student representatives.
2. **LMS API Access:** Secure API documentation and credentials; verify integration technical feasibility.
3. **Infrastructure Setup:** Provision cloud accounts (AWS/GCP); configure CI/CD pipelines; establish monitoring.
4. **Team Formation:** Recruit 6–8 developers (frontend, backend, DevOps, QA).
5. **Phase 1 Development:** Begin MVP implementation — target completion by end of Month 3.

8.4 Future Workshops

Workshop #3 — Implementation & Development: Hands-on development of Phase 1 (MVP); code review practices; Agile sprint planning with backlog grooming.

Workshop #4 — Testing, Deployment & Operations: Comprehensive testing execution; production deployment; monitoring setup; incident response procedures; post-launch optimization.

8.5 Continuous Improvement

Post-Launch Monitoring Cadence:

- Weekly: DAU, match conversion, group formation rate analytics.
- Monthly: User NPS and satisfaction surveys.
- Quarterly: External penetration testing.
- Bi-annual: Algorithm retraining on accumulated feedback data.

Feature Backlog: Peer skill endorsements · Video call integration · Advanced analytics for academic advisors · Gamification (badges, leaderboards) · Multi-institution support.

8.6 Acknowledgments

This design builds upon Workshop #1 systems analysis (Team 8, March 6, 2026), which identified critical barriers through primary survey data ($n = 25$) and comprehensive systems modeling. The team thanks Eng. Carlos Andrés Sierra, M.Sc., for guidance throughout the analysis and design phases.

References

References

[1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2015.

[2] D. Hardt (Ed.), "The OAuth 2.0 Authorization Framework," *IETF RFC 6749*, Oct. 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6749

[3] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[4] OWASP Foundation, "OWASP Top Ten 2021," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] G. Salton, A. Wong, and C. S. Yang, "A Vector Space Model for Automatic Indexing," *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, 1975.

[7] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2007.

[8] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[9] European Parliament and of the Council, "Regulation (EU) 2016/679 — General Data Protection Regulation (GDPR)," *Official Journal of the European Union*, Apr. 2016. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[10] J. Brooke, "SUS: A 'Quick and Dirty' Usability Scale," in *Usability Evaluation in Industry*. Taylor & Francis, 1996, pp. 189–194.

[11] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.

[12] K. S. Silva González, A. E. Casas Zapata, G. A. Beltrán Varela, and M. D. Tarazona Correa, "Workshop No. 1: Systems Analysis — Academic Collaboration Network," Systems Analysis & Design, Universidad Distrital Francisco José de Caldas, Mar. 2026 (unpublished).